

消除左递归: $A \rightarrow A\alpha | \beta$

扩展: $A \rightarrow A\alpha_1 | \dots | A\alpha_n | \beta_1 | \beta_2 | \dots$

(若)

$\Rightarrow \begin{cases} A \rightarrow \beta A' \\ A' \rightarrow \alpha A' | \epsilon \end{cases}$

$\Rightarrow \begin{cases} A \rightarrow \beta_1 A' | \dots | \beta_n A' \\ A' \rightarrow \alpha_1 A' | \dots | \alpha_n A' | \epsilon \end{cases}$

回溯 $\rightarrow$ 提取公因子

循环 $\rightarrow$ 消除左递归

$FOLLOW(A)$: 后继符号集

在某个句型中紧跟在A后的终结符号^q的集合

目前例子中, B后是C, 而 $C \rightarrow a, C \rightarrow c$, 故 $FOLLOW(B) = \{a, c\}$

$SELECT$ 选择集

$SELECT(A \rightarrow \beta)$ 是使用 " $A \rightarrow \beta$ " 时对应的输入符号的集合.

$SELECT(A \rightarrow \alpha\beta) = \{\alpha\}$

$(A \rightarrow \epsilon) = FOLLOW(A)$. (因为只有后继符号集不可能由A直接推出)

判断LL(1)? 先求 $FOLLOW, SELECT, FIRST$.

接着使用PPT图的方法判断

(左部相同, 对应的 $SELECT$ 不相交)

预测分析时先设是文法

LR分析. LR(0), LR(1)

一个栈, 一个保存编号, 一个保存串

每次根据栈顶+输入串符号确定归约到的串 (ACTION部分)

再根据栈顶编号+GOTO确定栈顶编号

ACTION 中终结符, GOTO 非~

rn 无脑归约 sn: 将符号a, 状态n压入栈



LR(0)分析表的构造

形式: $A \rightarrow \alpha_1 \alpha_2$ (有 $\cdot$)每个 α_i 都是一个闭包, 具体: 若 $S' \rightarrow \cdot S$ 在某个表中, 而规则中包含 $S \xrightarrow{A \cdot BC}$, 则 $S \rightarrow A \cdot BC$ 也在该表中由分析表构造: 状态 + $V_T = ACTION$ 若能到达下一个状态 n , 则为 S_n 若在结尾, 则用 rm, m 的规则规约

移进-规约

+ $V_N = GOTO$: 若能到状态 n , 则为 n

冲突:

例3. 某个状态为 $\begin{bmatrix} E \rightarrow T \cdot \\ T \rightarrow T * F \end{bmatrix}$, 则下一个字符为 $*$ 时, 可以规约(第一条), 可以移进(2)

语法规则

SDD, SDT (SDD: 产生式 + 属性, SDT: 产生式 + 语法规则写在旁边)

综合属性, 继承属性

S-属性、L-属性 (判断? 对综合属性无要求, 继承属性则看父亲、右兄弟, 自身其他继承属性但不成环)

若 L-属性的 SDD 文法可用 LL 分析, 则对应 SDT 可用 LL, LR

(判断可用 LL 分析? 看 SELECT 集合)

L-SDD 自顶向下翻译 (递归, 非递归)

每次取代最左的元素.

L-SDD 自底向上: 需要将 SDT 的每个规则移到结尾.

在中间的规则? $M \rightarrow \{ \dots \}$

类型表述:

类型 array 数组

pointer 指针

record 用结构体来描述结构体



中间代码生成

声明语句：注意用中间变量 t, w 指代临时的 $type, width$ ，等 $I \rightarrow BC$ 中 B 的类型确定之后再更新 $type, width$

数组用的赋值语句

$L \rightarrow id[E] | L[E]$

三地址码： $\begin{cases} t_1 = \dots (E + E | E * E) \\ t_2 = a[t_1] \rightarrow \text{注意这里不能使用 } base + t_1 \text{ 的写法!} \\ c = t_2 \end{cases}$ $a[t_1]$ 含义就是 $base + t_1$

多维数组寻址： $base + i_1 \times w_1 + i_2 \times w_2 + \dots$

w_1, w_2 都是对应的 $width$

如何组织数组？和普通变量类似，加一个指针，指向扩展属性（含：维数、各维宽度）

控制流的SDT (if-then-else)

几个核心的参数：条件 B 代码块 S

$B, true/false$ ：条件真/假跳转到的位置

$S, next$ ：执行完 S 跳转到的地方

while-do:

加一个 $B, begin$ ，因为循环执行完得回去

布局的语义分析

区分一下不同条件的 $true/false$ 属性（ $true/false$ 是继承属性）

控制流是 L-SDT，适合使用自顶向下的语法分析

控制流不是 LL 文法（判断：相同左部的 SELECT 相交），使用 LR 分析

三地址代码注意掌握！可能考

控制流注意一下是否带 fmm （是否加 fmm 优化）



注意一下如 $S \rightarrow S_1$ 的产生式中, S_1 指的是 S 的儿子, 但形式是一样的
形式上, 可能指代的是语法树的 S_1/S_1

将一个大条件语句进行中间代码生成时, 需使用回填技术

根据: 将条件内或若干基础的 $\text{if } a < b \text{ goto}$

将回填表 truelist , falselist 合并待度 (因为尚不知道跳转的位置)

特殊的, $\text{if } B_1 \text{ or } MB_2$ 中, 若 B_1 为 false , 需跳到 B_2 判断. 因此 B_1 为 false 是可以被更新的

访问链

是静态链

方向? 指向直接外围. 有多个? $\rightarrow$ 指向最近的

复习

图例:  A语言 A机器上运行的 L $\rightarrow$ A

常用的一种替换模式:  用 $L \rightarrow A$ 的翻译程序将 L 语言换成 A 语言

字母表闭包: $\Sigma^+ = \Sigma \cup \Sigma^2 \cup \dots$ 克林: $\Sigma^* = \{\epsilon\} \cup \dots$

文法的形式化定义: $G = (V_T, V_N, P, S)$ P : 产生式集合 S : 开始符号

句型: $S \Rightarrow^* \alpha$, $\alpha \in (V_T \cup V_N)^*$ α 为 S 的句型. (α 可为空串)

句子: $\alpha \in V_T^*$, α 为 G 的句子 (不含非终结符)

语言: 由文法 G 的开始符号 S 推导出的所有句子的集合称为 G 生成的语言

$$L(G) = \{w \mid S \Rightarrow^* w, w \in V_T^*\}$$



4种文法

0型: $\alpha \rightarrow \beta$ α 至少包含1个 V_N 1型: $|\alpha| \leq |\beta|$ 2型: $\alpha \in V_N$ ($A \rightarrow \beta$)3型: $A \rightarrow wB$ 或 $A \rightarrow w$ ($A \rightarrow Bw$ 或 $A \rightarrow w$) 分别为右、左线性文法逐级包含 $3 \subseteq 2 \subseteq 1 \subseteq 0$

CFG(2型文法)分析树

本从左到右的排列叶节点的符号串为这棵树的产出或边缘

短语: 分析树每一棵子树对应的边缘为短语

直接短语: 对应子树只有父子两代结点.

二义性: 文法可以为某个句子生成多棵分析树, 则为二义性的 (是对文法的概念)

(RE)

正则定义 均为 $d_i \rightarrow r_i$ 的形式每个 d_i 都是新符号, 不在字母表中, 各不相同每个 r_i 是字母表 $\Sigma \cup \{d_1, d_2, \dots, d_{i-1}\}$ 的正则表达式FA接收的语言: 对于一个串 x , 存在一个对应于 x 的转换序列, 则 x 被FA接收对于一个FA M , M 接收的所有串集合 $L(M)$ DFA/NFA $(S, \Sigma, \delta, s_0, F)$ 状态集合 $\downarrow$ 起始状态 $\rightarrow$ 终止状态集合区别 DFA: $S \times \Sigma \rightarrow S$ (一个点, 一个集合)NFA: $S \times \Sigma \rightarrow 2^S$ 带 ϵ -边的NFA: $S \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^S$

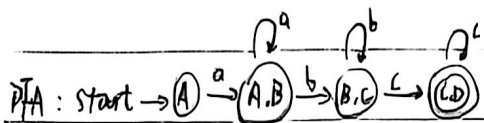
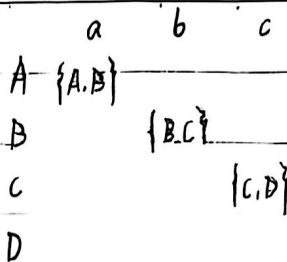
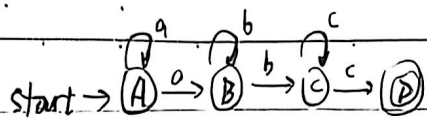
NFA和DFA等价. 如何转换?

DFA的每个状态都是一个由NFA中状态构成的集合.

写出转换表: 状态-输入.



Date.



错误处理: ① 看前缀是否有符合的 $\rightarrow$ 有? 继续 $\rightarrow$ 无? 处理错误 $\rightarrow$ 恐慌

② 恐慌模式: 输入中不匹配字符, 直到分析器在开头发现正确符号

词法分析识别: 单词、数字(如十、十六进制)、token(赋值号、数字)、注释

语法分析

最右推导(最右的推导) — 规范推导

最左推导 — 规范规则

预测分析: 是递归下降的特例, 在输入中看固定个符号来选产生式

向前看k个? LL(k)

* 递归下降可能回溯, 但预测分析不用

自顶向下文法

问题: ① 共同前缀: 回溯 $\rightarrow$ 提取左公因子, 如 $S \rightarrow aA|aB \Rightarrow \begin{cases} S \rightarrow aS' \\ S' \rightarrow A|B \end{cases}$

② 左递归文法会无限循环

消除直接左递归: $A \rightarrow A\alpha_1|A\alpha_2|\dots|\beta_1|\beta_2|\dots$ $A \rightarrow A\alpha|B$

$\Rightarrow \begin{cases} A \rightarrow \beta_1 A'|\beta_2 A'|\dots \\ A' \rightarrow \alpha_1 A'|\alpha_2 A'|\dots|\epsilon \end{cases}$ $\Rightarrow \begin{cases} A \rightarrow \beta A' \\ A' \rightarrow \alpha A'|\epsilon \end{cases}$

消除间接左递归: 如: $\begin{cases} S \rightarrow Aa|b \\ A \rightarrow Ac|Sd|\epsilon \end{cases} \Rightarrow \begin{cases} S \rightarrow bdA'|A' \\ A' \rightarrow cA'|adA'|\epsilon \end{cases}$

(把S代入试试)



推广公式: $A \rightarrow \alpha\beta_1 | \alpha\beta_2 | \dots | r_1 | r_2 | \dots$

$$\Rightarrow \begin{cases} A \rightarrow \alpha A' | r_1 | r_2 | \dots \\ A' \rightarrow \beta_1 | \beta_2 | \dots \end{cases}$$

递归下降 $\rightarrow$ 需回溯, 效率低. 不确定的分析

$\text{first}(X)$: 从 X 推导的串首终结符集合. 若 $X \Rightarrow^* \epsilon$, 则 $\epsilon \in \text{first}(X)$

计算: $X \rightarrow Y_1 \dots Y_k$ 中, 若 $\exists i, a \in \text{first}(Y_i)$, 则 $\epsilon \in \text{first}(Y_1) \dots \text{first}(Y_{i-1}) \Rightarrow a \in \text{first}(X)$

可选集 select 计算. 若 $\epsilon \in \text{first}(\alpha)$, 则 $\text{select}(A \rightarrow \alpha) = \text{first}(\alpha)$

$$\epsilon \in \text{first}(\alpha), \text{ 则 } \text{select}(A \rightarrow \alpha) = (\text{first}(\alpha) - \{\epsilon\}) \cup \text{follow}(A)$$

LL(1)文法 定义: $\forall A \rightarrow \alpha | \beta$ 满足: ① 不存在 $a \in V_T$, α, β 都能推导出以 a 开头的串.

② α, β 至多有一个能推导出 ϵ .

$$\beta \Rightarrow^* \epsilon, \text{ 则 } \text{first}(\alpha) \cap \text{follow}(A) = \emptyset$$

$$\beta \alpha \Rightarrow^* \epsilon, \text{ 则 } \text{first}(\beta) \cap \text{follow}(A) = \emptyset$$

} 同一非终结符的各个产生式的可选集互不相交.

L(从左到右) L(最左推导)

follow: 若 X 在结尾, 则 $\epsilon, \$$, 无 ϵ

计算方法: $A \rightarrow \alpha B \beta$, 则 $\text{first}(\beta)$ 除 ϵ 之外的符号放入 $\text{follow}(B)$ 中

$A \rightarrow \alpha B | A \rightarrow \alpha B \beta$, 且 $\text{first}(\beta)$ 含 ϵ , 则 $\text{follow}(A)$ 中所有符号都放入 $\text{follow}(B)$

预测分析 $\rightarrow$ 分为递归、非递归.

写出 select 集之后, 可以构造一个表: 非终结符 - 输入符号表 ("预测分析表")

非递归: 有一个栈, 存当前非终结符. 另一个寄存器存输入.

(4-1 PPT P8)

恐慌模式. 在转换表中加入非终结符的 follow 集 (synch 符号)

有 synch, 则弹出非终结符. 为空 $\rightarrow$ 忽略当前输入符号



预测分析步骤

① 消除二义性 (左递归, 左因子)

② 求 first, follow $\rightarrow$ 求 select

③ 检查是否为 LL(1) 文法 $\rightarrow$ 构造预测分析表

移入-归约分析中: 栈内符号串 + 剩余输入 = 规范句型

LR: 自底向上语法分析

给 ACTION, goto 表可用来进行语法分析 (LR 分析器)

每一项内容为: sn/rn sn : 状态 n 压入栈 rn : 用第 n 个产生式规约

增广文法: 加入 $S' \rightarrow S$ (使开始符号只有 1 个)

构造 LR(0) 自动机

先将产生式加入图点, 然后求 "新项" (即闭包)

$\begin{cases} A \rightarrow \alpha \cdot \beta & \text{移进项目} \\ A \rightarrow \beta & \text{归约项目} \end{cases}$

闭包 $\rightarrow$ 若 $A \rightarrow \alpha \cdot \beta \in \text{closure}$, 则 $B \rightarrow \gamma \in \text{closure}$

$\text{goto}(I, X) = \text{closure}(\{A \rightarrow \alpha X \cdot \beta \mid A \rightarrow \alpha \cdot X \beta \in I\})$

可以利用闭包构造 LR(0) 分析表 P_{41} .

问题: LR(0) 可能存在移进-归约冲突和归约-归约冲突.

$\downarrow$
如: LR(0) 表中同一项可能同时有 rn, sn

不是所有 CFG 都能用 LR(0) 方法分析

词法分析器结果: 单词种别编码, 自身值

输入/加对象: 源程序

语法分析器输入: Token 对象

ϵ 可在 first 中 $\$$ 可在 follow 中



$S \rightarrow A \cdot aB$ 移进

$S \rightarrow \cdot bBB$ 移进

$S \rightarrow b \cdot BB$ 待约

$S \rightarrow PbB \cdot$ 归约

{有圆点, 都称为“项目”}

LR分析栈内容的特征: 是规范句型的前缀

(分析栈内容 + 剩余输入 = 规范句型)

可归约符号串——“句柄”

终结符可以有综合属性, 是由词法分析器提供的词法值。

(无继承)

S-SDD 可使用 LR 分析技术, 则 SDT 可在 LR 分析过程中实现

(此处无 LL, 因为 S-SDD 中, 所有 SDT 的语句都在结束位置)

L-SDD

LL

LL/LR

在非递归的预测分析中翻译, 需扩展语法树

综合属

{综合属性存放在 A 本身记录中 X Asyn!}

语法树翻译可从上到下, 可从下到上

目标代码生成

与中间代码生成无关的是: 便于将语言的组织

类型表达式: 基本类型, 类型名都是类型表达式

CFG 不一定是 LR 的文法

活前缀: 不含句柄右侧任意符号的前缀

即: abcde, 则 abcde 不是活前缀
句柄



LR(1) 右部定尾符号方法: $[S \rightarrow \alpha AB, a] \Leftrightarrow [A \rightarrow \cdot r, b]$
 $a \in \text{first}(AB) \quad b \in \text{first}(BA)$

LR(1) 自动机书写形式

$I_0: S' \rightarrow \cdot S, \$$

对如何求闭包: $[S \rightarrow \alpha \cdot AB, a] \Leftrightarrow [A \rightarrow \cdot r, b]$

$b \in \text{first}(BA) \quad \beta = \epsilon? \quad b \in \text{first}(a) = a$

求 GOTO 时, 只有对应尾符号才能判约

LALR

同心集: 2 个 LR(1) 项目只有尾符号不同

合并同心项集不会产生移进/归约的冲突 (因为 2 个是同心集, 无论是移进/归约对应之前/后的操作都啊)

分析能力 不会推迟错误的发现

$LR(0) < SLR < LALR < LR(1)$

可能有归约/归约冲突

二义性文法都不是 LR 的

四元式序列

$(+, b, c, a) \quad a = b + c$

原代码 $a = b + c$

$(=, [], y, i, x) \quad x = y[i]$

> 注意这里的变量 x 宽度! 如 4 个字节, 则 $i = 4 \times t$

$([, y, x, i) \quad x[i] = y$

$a[i]$

$(\text{param}, -, -, t) \quad (\text{call}, f, l, t_2) \quad t_2 = f(t)$

$(\&, y, -, x) \quad x = \&y$

$(*, y, -, x) \quad *x = y$

$(=, *, y, -, x) \quad x = *y$

while 普通译三地址码别忘了: 基本块一定有 goto 开头 (就算不执行也要加)

函数调用普通译: $\text{param } x, \text{param } y, \text{call } f_1$

2 个参数



画SDO/SDI → 用增广文法!

动态存储分配: 编译时, 只产生各种必要的信息

运行时, 动态分配数据对象的存储空间

过程的每一次执行称为活动

分配一段连续存储区用于管理信息, 称为活动记录

调用、被调用之间传递值一般被放在被调用者的活动记录开始位置

访问链: 先求出嵌套深度 (最外层函数深度为1) (编译过程可确定)

对于 n_x (深度) 调用 n_y

若 $n_x < n_y \rightarrow n_y = n_x + 1$. 直接访问链 $n_y \rightarrow n_x$

若 $n_x = n_y \rightarrow n_y$ 连 n_x 连的那个结点

若 $n_x > n_y \rightarrow n_y$ 连最近的 $n_y - 1$ 结点

display: d[i] 连向控制栈最近的深度为 i 的点

有一个进栈: 在此结点的 d[i] 连向 display d[i] 指向的点

display d[i] 指向新点

参数传递

传值、地址

传值结果 (甲先传值, 最后写回对应变量)

传名 → 类似宏定义, 直接替换

符号表

可能会连向外部符号表 → 内情向量



Date.

所有变量地址之和

符号结构

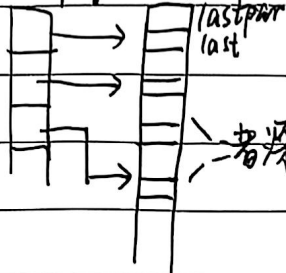
header	
名称	type

→ 此变量的地址. 如 0 (起始地址)

符号表另一种组织形式

display 表 + lastpar (最后一个链接地址) + last (最后一个名字, 包括函数)

display btch



nametab

name	link

→ 第一个名字, 则为 0
否则指向当前层上一个名字

编译器使用 说明标识符的过程或函数名区别标识符作用域

$[A \rightarrow \alpha \cdot B \beta, a] \leftrightarrow [B \rightarrow \gamma \cdot \delta, b]$

$b \cdot \beta \Rightarrow^+ \epsilon, b = a$ 称为继承的右继符, 否则称为自生的

CFG 上 LR 文法

二义性文法不是 LR 的

一定不是

句柄: 句型最左直接短语

活前缀: 规范句型的前缀, 右句柄右侧无符号

while

if $c \leq 5$ then while $x > y$ do $z = x + 1$ else ..

这个结束之后, 需 + goto 语句回到外层循环

$t = a[i]$

$x = t$



扫描全能王 创建